

DAHE: Parameter-Adaptive and Memory Efficient FPGA Acceleration of Homomorphic Encryption

Yilan Zhu¹, Honghui You¹, Wei Zhang¹, Jiming Xu², Qian Lou³, Shoumeng Yan³, and Lei Ju⁴

Abstract—While homomorphic encryption (HE) has been well-recognized as a promising data privacy protection technique, there are many challenges to the real-world deployment of HE applications. In this work, we propose a design flow for parameter-adaptive and memory-efficient FPGA acceleration of homomorphic encryption. In the framework, we explore the correlations between HE parameter selection to meet various design objectives and the huge design space due to underlying FPGA hardware resource allocation. Particularly, we demonstrate that adaptive management of the FPGA memory hierarchy is crucial to supporting diverse cryptosystem parameter selection for application-level security, accuracy, and performance requirements. We propose a resource-efficient and flexible micro-architectural design for HE operations, where data access patterns in various pipeline execution stages are optimized for high memory bandwidth utilization. Furthermore, a memory-aware performance model is built for automatic design space exploration for cryptosystem parameter selection and hardware resource provisioning. Experimental results show 1.50X and 1.16X speedup for the NTT and Rotation operations w.r.t. the state-of-the-art FPGA implementation. Meanwhile, the proposed framework generates flexible and high-performance accelerator code for real HE application kernels with different cryptosystem parameters on a wide range of FPGA devices.

Index Terms—Homomorphic encryption (HE), FPGA, high-level synthesis, design space exploration, memory management.

Received 9 July 2024; revised 25 February 2025; accepted 25 April 2025. Date of publication 22 May 2025; date of current version 11 July 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62372272; in part by the Shandong Provincial Natural Science Foundation under Grant ZR2024MF099 and Grant ZR2023LZH014; in part by the Department of Science & Technology of Shandong Province under Grant SYS202201; in part by Quan Cheng Laboratory under Grant QCLZD202302; and in part by the Taishan Scholars Program under Grant tsqn202211281. Qian Lou contributed independently to this work, without support from the aforementioned funding sources. Recommended for acceptance by M. Correia. (Corresponding author: Lei Ju.)

Yilan Zhu is with the School of Cyber Science and Technology, Shandong University, Qingdao 266237, China, and also with Ant Group, Hangzhou 310000, China.

Honghui You and Wei Zhang are with the School of Cyber Science and Technology, Shandong University, Qingdao 266237, China, and also with Quan Cheng Laboratory, Jinan 250103, China.

Jiming Xu and Shoumeng Yan are with Ant Group, Hangzhou 310000, China.

Qian Lou is with the University of Central Florida, Orlando, FL 32816, USA.

Lei Ju is with Quan Cheng Laboratory, Jinan 250103, China (e-mail: julei@sdu.edu.cn).

Digital Object Identifier 10.1109/TC.2025.3569159

I. INTRODUCTION

THE prevalence of data breaches in cloud computing underscores the paramount importance of data protection. Fully homomorphic encryption (FHE) allows computations to be performed directly on encrypted data without decryption, enabling secure offloading of computational tasks to third-party servers. FHE stands out as a forward-looking technology with significant potential for data computing privacy. However, a key hindrance to the widespread use of FHE is the extremely high computational cost of the encrypted data [1], [2]. FHE schemes can be up to five orders of magnitude more demanding in terms of computation and storage than operations on unencrypted data. For example, CryptoNets [3] increases neural network inference latency from milliseconds to hundreds of seconds due to the overhead rooted in the security-focused design of FHE schemes.

Therefore, considerable research is currently focused on FHE accelerators [1], [2], [4], [5], [6], [7], [8]. Supporting a range of parameter settings is crucial in the design of accelerators, as the selection of cryptosystem parameters directly affects the deployment of homomorphic encryption by influencing both security and computational precision [9]. It is important to note that these parameters are also closely related to the data volume processed and the required computation time. Lou et al. [10] indicate that opting for smaller parameters can significantly boost performance while trade-offs in security and accuracy remain within acceptable limits for practical applications.

In accelerator design, FPGA-based hardware accelerators for FHE have recently attracted much attention due to FPGAs' programmability and adaptability. However, most literature on FPGA-based HE accelerator design focuses on tailoring specific cryptosystem parameters to a particular FPGA architecture [6], [11]. Meanwhile, the compelling need for data privacy has spread across data-centric applications widely deployed on platforms from low-power IoT infrastructures to high-performance data centers. Diversified HE application scenarios impose different cryptosystem parameter settings and hardware configurations [12], [13], [14]. This paper tackles the following design challenges for FPGA-based HE accelerators.

Challenge 1: Subtle alterations of HE parameters may lead to significant changes in the FPGA's resource usage [15]. An FPGA accelerator needs to be flexible to support distinct HE parameter settings while balancedly utilizing different types of hardware resources (e.g., LUTs and DSPs).

Challenge 2: In many existing designs, the level of parallelism for memory-intensive HE computation is often restrained by the limited size of on-chip memory [16]. However, there remains a problem with conducting joint optimization between on-chip and off-chip memories in a sophisticated pipeline design to improve the level of parallelism and overall performance.

Challenge 3: For the agile development of FPGA-based FHE accelerators, it is crucial to build a design automation framework to explore the joint impact of HE parameter selection on design objectives and hardware resource utilization.

In this paper, to overcome the above challenges, we propose **DAHE**, a parameter-adaptive and resource-efficient design automation framework for FPGA-based HE acceleration. First, we propose low-level optimization schemes for HE operations that balance computation parallelism, hardware resource usage, and on-/off-chip memory bandwidth utilization. Then, we build an estimation model that captures the correlation between HE parameters and resource/memory usage of individual HE kernels. Finally, for given application-level design objectives and specification of target FPGA devices, the framework automatically performs design space exploration (DSE) to identify compliant HE parameters and resource-efficient accelerator circuit deployment. The contributions of this work are summarized as follows:

- We optimize the basic HE operations to support parameter-adaptive and resource-efficient design automation. In particular, our modular reduction implementation balances the system-level resource usage between DSPs and LUTs. The proposed parallel automorphism kernel allows for scalable memory bandwidth utilization in the pipeline design of high-level HE operations.
- We propose a memory management scheme for on- and off-chip memory co-allocation and optimization. We model the correlation among HE parameters, data access patterns of individual kernels, and bandwidth utilization in the memory hierarchy to generate optimized data layout and off-chip data transfer schemes.
- For given application-level design objectives (including performance, security, and approximation error) and target FPGA devices, the proposed DSE framework produces a set of Pareto optimal solutions containing compliant HE parameters and deployment configurations (e.g., parallelism of HE kernel, memory layout, etc.)
- We adopt the Pareto-optimal HE parameters and deployment configurations as data declarations and directives in an FPGA high-level synthesis (HLS) toolchain to generate the accelerator bitstreams. The agile development environment significantly improves design flexibility and time-to-market.
- DAHE supports a wide range of application-level HE parameters and FPGA devices, from low-power embedded devices to high-end data center accelerator cards. Compared with the state-of-the-art HE accelerator implemented in Verilog, the proposed framework achieves a 1.50X speedup for the NTT operations. Additionally, the framework facilitates flexible deployment of homomorphic neural network applications.

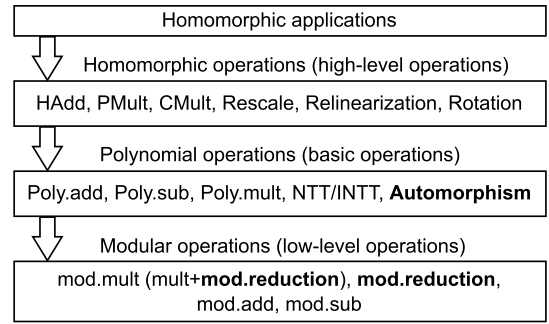


Fig. 1. Hierarchy of homomorphic operations.

II. BACKGROUND AND RELATED WORK

A. FHE Scheme

Fully Homomorphic Encryption (FHE) schemes, such as BGV [17], BFV [18], TFHE [19], and CKKS [20], have been developed for various applications. Among them, CKKS [20] supports fixed-point arithmetic on ciphertexts, enabling versatile real-world use cases. This work presents an accelerator framework specifically optimized for CKKS.

In the CKKS scheme, a vector m is encrypted into a ciphertext represented by a pair of polynomials, denoted as $\text{ct}(m) = (c_0, c_1)$. Both polynomials c_0 and c_1 belong to the polynomial ring $R_{Q,N}$. To ensure security, the coefficient modulus Q for the polynomial ring's elements ranges from several hundred to several thousand bits, and the polynomial degree N is a power of two and often exceeds 2^{12} .

Due to the increased computational complexity caused by large homomorphic parameters, the Residue Number System (RNS) scheme has been proposed for efficiently processing large-number arithmetic in the mod Q domain. This method decomposes polynomial coefficients with having a bit width of $\log_2 Q$ into smaller moduli, denoted as q , where $Q_l = \prod_{i=0}^{l-1} q_i$. Here, the parameter l represents the ciphertext level, dictating the depth to which multiplication processes can be consecutively executed. Following the decomposition, the ciphertext transforms from two polynomials modulo Q into $2l$ polynomials modulo q_i , known as RNS polynomials, facilitating parallel computations. Moreover, CKKS boosts efficiency with vector encryption, enabling SIMD-like operations across vector slots. Although this supports thousands of elements per ciphertext, inter-slot computations demand complex rotations, increasing computational overhead.

B. Homomorphic Operations

Homomorphic operations are a critical component of homomorphic encryption schemes, with their hierarchical structure illustrated in Fig. 1. We define the ciphertext as $\text{ct} = (c_0, c_1)$, and l is the level of ciphertext. Plaintext is denoted as $\text{pt} = p_0$. Below is a concise introduction to the homomorphic operations along with their computational complexities based on the count of polynomial operations:

- **HAdd**(ct, ct') $\rightarrow \text{ct}_r$: Performs ciphertext addition, yielding $\text{ct}_r = (d_0, d_1)$, where $d_0 = c_0 + c'_0$ and $d_1 = c_1 + c'_1$. Complexity: $O(l)$.

- **PMult**(ct, pt) $\rightarrow$ ct_r: Multiplies a ciphertext by a plaintext, resulting in ct_r = (d₀, d₁), where d₀ = c₀ · p₀ and d₁ = c₁ · p₀. Complexity: O(l).
- **CMult**(ct, ct') $\rightarrow$ dt_r: Multiplies two ciphertexts, producing dt_r = (d₀, d₁, d₂), where d₀ = c₀ · c'₀, d₁ = c₀ · c'₁ + c₁ · c'₀, and d₂ = c₁ · c'₁.
- **Keyswitch**(d₂) $\rightarrow$ (d₀, d₁): Adjusts decryption keys via steps: 1. Compute d'₂ = ModUp(d₂). 2. (d'₀, d'₁) = d'₂ · evk. 3. (d₀, d₁) = (ModDown(d'₀), ModDown(d'₁)). ModUp and ModDown use base conversion BConv. The “evk” means evaluation keys, and evk $\in R_{OP}^{2(l+k)}$. The BConv is computed as BConv(a)_{C $\rightarrow$ D} = $\left(\sum_{j=0}^{l-1} [a^{(j)} \cdot \hat{q}_j^{-1}] \cdot \hat{q}_j \bmod p_i\right)_{0 \leq i < k}$, where C and D are sets of prime moduli for q_j (0 ≤ j < l) and p_i (0 ≤ i < k), respectively. a^(j) denotes the RNS representation modulo q_j. Complexity: O(l(l+k)).
- **Relinearization**(dt) $\rightarrow$ ct_r: Reduces polynomials after CMult from 3 to 2, producing ct_r = (d₀, d₁) + Keyswitch(d₂, evk). Complexity: O(l) (excluding Keyswitch).
- **Rescale**(ct) $\rightarrow$ ct_r: Reduces the scale factor Δ and removes the last prime modulus from the modulus chain, yielding ct_r = q_l⁻¹(ct - ct''), where ct'' = ct mod R_{Q_{l-1}}. Complexity: O(l).
- **Rotation**(ct, s) $\rightarrow$ ct_r: Cyclic shifting message vector elements by s slots. Steps: 1. c'₀ = automorphic(c₀, s), c'₁ = automorphic(c₁, s). 2. (c''₀, c''₁) = Keyswitch(c'₁, evk). 3. ct_r = (c''₀, c'₁ + c''₁). Complexity: O(l) (excluding Keyswitch).

Homomorphic operations can be decomposed into basic operations on RNS polynomials, primarily polynomial addition and multiplication. Given the high complexity of polynomial multiplication at O(N²), the Number Theoretic Transform (NTT) reduces it to O(N log N). This operation converts coefficient representations to point-value forms through fast interpolation, performs vector multiplication at O(N), and the Inverse NTT (INTT) then reconstructs the polynomial coefficients. This work focuses on designing the following two HE operations to support high-level resource provisioning and design space exploration.

Automorphism is a basic operation for rotation, permuting the coefficients of the ciphertext RNS polynomial to adjust the positions of the underlying unencrypted data [21]. This permutation introduces challenges for parallel computation. Our implementation, based on the SEAL library [22], performs automorphism in NTT form. Since the input of the NTT is in normal order the output is in bit-reversed order, and the INTT follows the reverse process, automorphism performed after NTT and before INTT must handle data in bit-reversed order with point-value representation. Compared to traditional automorphisms on coefficient representations in normal order, its primary overhead lies in the additional data rearrangements, making the computation more complex. Since data in SEAL is stored in the format after NTT operation, NTT-form automorphism is thus widely employed. The NTT-form automorphism is described in Algorithm 1.

Modular arithmetic serves as a foundational operator for basic operations, encompassing modular addition, subtraction, and multiplication. Of these, modular multiplication—comprising

Algorithm 1: Automorphism in NTT-form.

Input: a[N], galois_elt (parameter determined by rotation slots count s).

Output: $\bar{a}[N]$

```

1: m ← 2 · N − 1
2: for i ← 0 to N − 1 do
3:   r ← reverse_bits(i)
4:   raw ← galois_elt · (2 · r + 1)
5:   raw ← (raw & m − 1) ≫ 1
6:   index ← reverse_bits(raw)
7:    $\bar{a}[i] \leftarrow a[index]$ 
8: end for

```

multiplication followed by modular reduction—is notably the most intricate. Consider the multiplicands a₁, a₂, and the modulus q each a k-bit number. Multiplying a₁ by a₂ yields a 2k-bit product. Directly calculating a mod q using traditional division is computationally demanding. To enhance efficiency, techniques such as Barrett [23] and Montgomery [24] reductions are employed, with Barrett reduction being notably prevalent in FHE algorithms. Barrett reduction streamlines the modular reduction process by using a pre-computed parameter for performing two multiplications, followed by a conditional check to obtain the final result.

C. Related Work of FHE Accelerators

Hardware acceleration for FHE has recently gained significant attention. Intel’s HEXL [25] enhances CPU performance 6 to 7 times by optimizing HE’s modular arithmetic and NTT with Advanced Vector Extensions 512 (AVX-512). GPU solutions [4], [8], [26] focus on maximizing the robust parallel processing capacity by tailoring HE operations and data access patterns to complement the GPU’s computational and memory access traits. ASIC designs [1], [2], [5], [27] significantly boost overall performance with thousandfold enhancements. F1 [1] was the first ASIC-based work to implement FHE bootstrapping. Craterlake [27] proposed a design with higher parallelism for functional units to improve performance. A recent work SHARP [5] focuses on the design of architectures with shorter word lengths and area optimization. However, the limited reconfigurability of ASIC, with fixed HE functional units and batch-processing element quantities, prohibits adaptability to fast-evolving algorithm-level FHE optimizations (e.g., [28], [29]). FPGA is a promising candidate for FHE acceleration [6], [7], [11], [16], [30], [31]. HEAX [16] optimized the critical HE operations like NTT and KeySwitch, using elaborate pipeline and parallel execution designs. Poseidon [7] introduced an NTT fusion scheme that decreases computational stages and processing time. coxHE [32] provides a resource-performance model on low-power FPGA devices, where the model does not explicitly consider the memory hierarchy. Medha [11] has developed a fine-grained ciphertext splitting method and a corresponding low-level FPGA architecture to accommodate multiple HE parameter settings. Many studies [1], [33] have revealed that

memory bandwidth is a critical bottleneck for FHE acceleration. MAD [34] introduced a generalized on-chip memory management scheme for HE accelerators to minimize off-chip DDR accesses.

III. MOTIVATION

A. Basic Operations Acceleration

Among low-level operations in homomorphic computing, modular reduction stands out due to its high frequency of use (exceeding 50%) and significant computational cost (requiring multiple multiplications), with usage varying by scheme. Traditional Barrett reduction primarily relies on DSPs. As data bit-width increases, DSP requirements grow substantially. Conversely, Will and Ko's reduction [35] involves multiple iterations of additions and shifts. The scheme does not use DSP but requires more LUTs. We implemented these schemes, observing that Will and Ko's reduction's LUT use is nearly fourfold Barrett's, while Barrett's DSP usage escalates from four at 27 bits to eighteen at 54 bits. Considering the limited on-chip computational resources for deploying homomorphic operations on FPGAs, excessive use of either DSPs or LUTs can limit the number of configurable operators, thus impacting overall performance. Therefore, we need to explore an optimized modular reduction strategy that balances computational resources like DSPs and LUTs to enhance performance.

Meanwhile, automorphism is a critical basic operation that significantly impacts pipelining efficiency within rotations, frequently used in homomorphic applications. Without optimization, the cycles of automorphism consume 62% of the rotation pipeline interval, severely degrading its performance. If we enhance internal parallelism and reduce cycle counts in automorphism, the pipeline interval for rotations could be halved. However, as automorphism heavily relies on memory resources, increasing parallelism also raises memory resource consumption. Therefore, designing a parallel computation strategy for automorphism and coordinating its memory usage through DSE significantly impacts the performance of rotation operations and the overall FPGA deployment.

B. Limited On-Chip Memory Capacity

Literature works [1], [6], [33] have devised approaches to leverage on-chip memory effectively for performance enhancement. However, the limited on-chip memory capacity typically becomes a resource bottleneck in the overall system design. For instance, FAB [6]'s design employs over 95% of BRAM/URAM resources while using only 56% of DSP resources of a U280 FPGA board, demonstrates an unbalanced utilization of compute and on-chip memory resources.

We further demonstrate the on-chip memory limitation on a low-power Xilinx ZCU102 device using a preliminary implementation of HE matrix multiplication with parameters $N = 8192$, $L = 3$, and 54-bit small prime moduli. When attempting to parallelize 8 computing cores within the NTT, it requires 718 DSPs (20%) and 2560 BRAM blocks (140%, over budget). Therefore, this paper explores the possibility of relocating non-essential

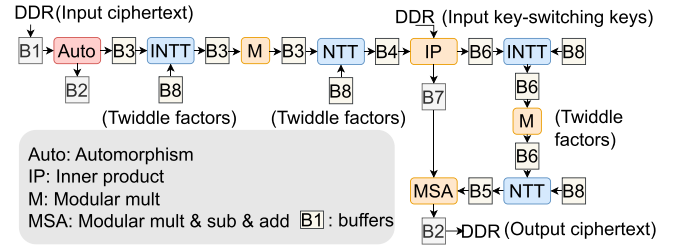


Fig. 2. Rotation computation process.

on-chip buffers to off-chip memory and then repurposing the freed on-chip memory for additional operators.

We present a motivating example to show the impact of placing data buffers to on-chip or off-chip memories on the overall performance of the expensive Rotation operation. The computational process and buffer usage of Rotation is shown in Fig. 2. We deploy the operation on a Xilinx ZCU102 FPGA with homomorphic parameters set to $N = 8192$, $L = 4$ and $\log_2 q = 32$. Given the limited capacity of on-chip BRAMs on the ZCU102, not all buffers can be allocated on-chip. Fig. 3(1) shows the performance gain of individual basic operations, including NTT, Automorphism, and MSA (modular mult/add/sub) when corresponding buffers are allocated to the on-chip BRAMs (normalized to the performance with off-chip buffer allocation). Compared with the huge performance gain for NTT with “butterfly” accesses and Automorphism with non-sequential accesses, MSA operations with burst access patterns can be placed to off-chip memory with small performance loss (e.g., 14%) while saving limited on-chip memory resources.

To show the impact of off-/on-chip co-allocation on the overall performance of Rotation, we evaluate 8 different buffer allocation schemes. Subject to the capacity limitation of on-chip BRAM, we alternatively place 1 out of 8 buffers off-chip while keeping the rest on-chip (e.g., “off-B1” placing only B1 off-chip). Fig. 3(2) shows the Rotation performance under different schemes. The performance drops more than 10-fold when NTT and automorphism buffers are moved off-chip (from “off-B3” to “off-B6”), Schemes “off-B2” and “off-B7” suffer less performance loss w.r.t. the theoretical bound of 4833 op/s (simulated results that assumes all buffers are placed on-chip). Note that placing B8 with a butterfly access pattern off-chip incurs less performance loss than other NTT buffers, primarily due to the small data access volume for the twiddle factors in the initial stages of NTT, which allows them to be stored in registers for repeated access.

In conclusion, the illustrative example suggests that placing certain buffers on off-chip DRAMs may have less impact on overall performance. Meanwhile, it may lead to better utilization of on-chip memory with a higher level of computation parallelism. In a practical accelerator design, the on-/off-chip data co-allocation produces a large design space, and design automation techniques are of paramount importance.

C. Off-Chip DDR Bandwidth Utilization

With the above-mentioned memory co-allocation scheme, off-chip DDR bandwidth utilization become a new challenge.

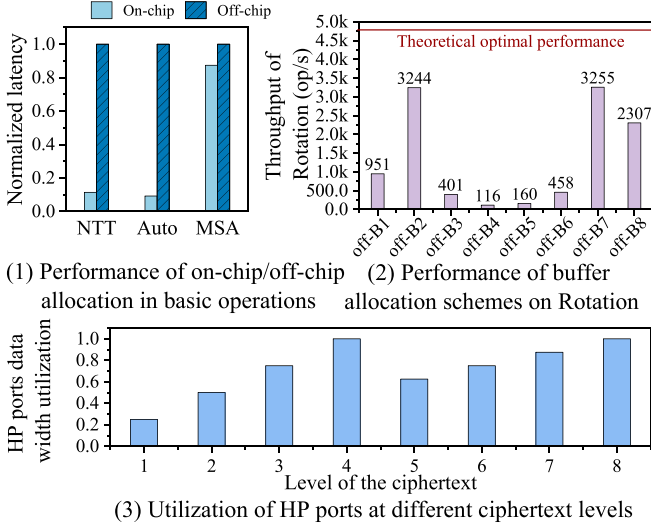


Fig. 3. Effects of buffer allocation to off-chip memory on HE operation performance.

Our initial investigation shows the under-utilization of DDR bandwidth when accessing data off-chip via High-Performance (HP) ports. Typically, an HP port has a width of several hundred bits, whereas the bit-width of homomorphic ciphertext data usually ranges between 20 to 60 bits (dependent on the selection of q). Therefore, concatenating multiple data entries to match the HP port's width is a promising strategy for improving bandwidth utilization. However, maintaining high utilization throughout the homomorphic application is challenging. If data from the same ciphertext are concatenated based on identical coefficient indices of each RNS polynomial, the varying number of RNS polynomials during computations prevents consistent, efficient use of the HP port's bandwidth. This issue is illustrated in Fig. 3(3), which shows the bit-width utilization for different numbers of RNS polynomials (equivalent to ciphertext level L). Thus, alternative concatenation strategies are necessary. Given that concatenated data must be immediately processed in parallel after being fetched according to the off-chip buffer scheme, the strategy must align with operations that can be computed in parallel.

D. DSE for Multi-Objective Optimization

In this section, we show how homomorphic parameters influence buffer allocation and highlight the need to explore the design space to optimize hardware configuration accordingly. Table I displays two sets of HE parameters for a Rotation operation, along with feasible hardware configurations on a Xilinx ZCU104 platform. For the small HE parameter set, configuration X1, X2, and X3 deploy 4, 8, and 16 parallel NTT cores, respectively. When the parallel NTT cores increase from 4 to 8, more on-chip memory usage is required to provide double on-chip memory bandwidth (due to the dual-port design of BRAM blocks). As a result, other buffers of the Rotation operation must be placed off-chip subject to the on-chip memory capacity. Even with the proposed data layout management and HP port allocation, the overall throughput of Rotation improves significantly with the increased parallelism of NTT and other

TABLE I

PERFORMANCE COMPARISON OF BUFFER ALLOCATION AND HOMOMORPHIC PARAMETER CHOICES FOR HE ROTATION ON ZCU104 FPGA, WHERE "HP" INDICATES THE NUM OF HP PORTS USED, AND "P" REFERS TO THE NUMBER OF PARALLEL NTT CORES DEPLOYED

Parameter			Cfg	P	On-/Off-Chip Buffers(%)	BRAM/URAM Usage	HP	Perf. (op/s)
N	$\log q$	L						
2^{12}	32	3	X1	4	88/12	464/52	3	6103
			X2	8	68/32	576/68	4	12207
			X3	16	44/56	576/68	4	4308
2^{14}	36	4	Y1	2	82/18	512/88	3	572
			Y2	4	34/66	512/54	4	1144

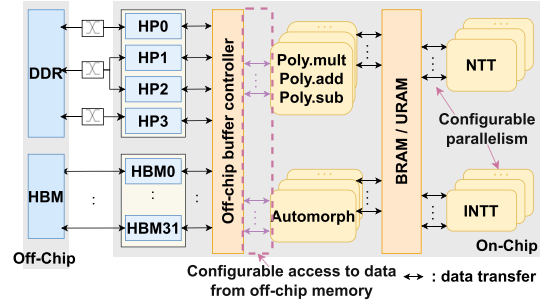


Fig. 4. Architecture overview of the proposed FPGA-based HE acceleration.

basic operations. However, further increase in the parallelism slows down the overall performance of Rotation because (i) it is not possible to ensure NTT cores' single-cycle data access latency due to limitations on several BRAM blocks and (ii) increased off-chip data traffic exceeds the available DDR bandwidth.

Similarly, observations can be made for the large HE parameter set. Meanwhile, it can be observed that the HE parameter greatly impacts the cipher-text volume and memory allocation scheme. The illustrative example reveals that HE parameter setting and FPGA hardware resource availability affect optimal buffer allocation and memory bandwidth utilization. With joint consideration of HE parameter setting's impact on the security level and computational accuracy, finding the optimal solution for hardware configurations and parameter selection through automatic design space exploration is the main goal of the proposed framework.

IV. ACCELERATOR ARCHITECTURE

A. Overall Architecture

The overall architecture of the proposed Design Automation framework for HE acceleration (DAHE) is depicted in Fig. 4, comprising computation units, on-chip BRAM/URAM blocks, and off-chip memories (DDR and/or HBM), as well as an off-chip memory controller associated with access ports. In the computation unit, we implement basic operations for processing RNS polynomials, including Automorphism, NTT/INTT, and modular arithmetic. Since operations like modular addition and multiplication involve sequential access, they can be fused in specific homomorphic operations. In executing homomorphic

operations, we design pipeline and parallel processing mechanism among basic operations, following the approaches of [16], [32]. For hardware implementation, we explore the design space by adjusting the intra-parallelism of basic operations, i.e., the number of low-level operators, to balance resources and performance. As parallelism further affects the number of data channels connecting to on-chip or off-chip memory, the data channels are also configurable to match the bandwidth of the computation units.

Regarding memory hierarchy, our primary design strategy involves storing the buffers necessary for NTT and INTT operations on-chip memory. The assignment of buffers for other operations, whether on-chip or off-chip, is determined by exploring the design space, considering the data size of homomorphic applications and FPGA specifications. This approach seeks to balance utilizing off-chip bandwidth and on-chip capacity. The off-chip buffer controller is primarily responsible for allocating HP and HBM ports (if the device is equipped with HBM) and scheduling different operations access to maximize bandwidth. This architecture integrates optimizations for memory management and homomorphic operations. Through design space exploration, it achieves performance advantages while meeting homomorphic applications' security and accuracy requirements.

B. Basic Operations Optimization

In homomorphic computing, modular reduction demands considerable computational resources, and automorphism suffers from heavy memory traffic. It is essential to support adaptive HE parameter selection, as it significantly impacts the overall performance and design space. This section focuses on optimizing modular reduction and Automorphism operations.

1) *Modular Reduction Acceleration*: For the modular reduction, we propose an optimized algorithm that reduces DSP resource consumption in FPGAs based on Barrett reduction [23]. The algorithm requires two multiplications, with the DSP resource demand increasing substantially as the bit width expands. To reduce the usage of DSP, we leveraged the characteristic of a limited number of small prime moduli in homomorphic encryption parameters, extracting the effective bits of the prime modulus and thereby reducing the data bit width in actual multiplication operations. We integrated this approach shown in Algorithm 2, optimizing both multiplication operations in Barrett reduction. When the data bit width is set to 54 bits, which is favorable for DSP specifications, it can reduce the DSP usage from 18 to 4 units; at a bit width of 32 bits, which is advantageous for HP port transmission, this method can reduce the DSP usage from 7 to 5 unit.

In our approach, we process the modulus q and its precomputed value μ , and subsequently derive i and j to reduce the bit width. This approach restricts the range of moduli values, which must satisfy the formula $q = 2^k - 2i \cdot 2^N + 1$, where k denotes the bit width of q , N denotes the degree of the polynomial, and i is the factor used to generate the prime modulus. This restriction arises from the parameter selection in homomorphic encryption schemes [36]. In addition, the number of prime moduli generated by the formula increases as the bit width of q grows. When

Algorithm 2: Optimized Barrett Modular Reduction.

Input: integer a , modulus q , bitwidth of q is k , and the polynomial degree is N

Output: $a \bmod q$

Precompute: i , where $q = 2^k - i \cdot 2^{N+1} + 1$; $\mu = \lfloor \frac{2^{2k+1}}{q} \rfloor$; j , where $j = \mu - (2^{k+1} + 2(i \cdot 2^{N+1} - 1))$

- 1: $a' \leftarrow a[:(k-2)]$ $\triangleright$ Extracts the bits from $k-2$ to the most significant bit
 - 2: $t1 \leftarrow ((a' \cdot i) \mid 0^{N+1}) - a'$ $\triangleright$ Appends 0^{N+1} to the right of the product of a' and i
 - 3: $t2 \leftarrow a' \cdot j$
 - 4: $t3 \leftarrow (a' \mid 0^{k+1}) + 2 \cdot t1 + t2$
 - 5: $t4 \leftarrow t3[:(k+3)]$
 - 6: $t5 \leftarrow (t4 \cdot i) \mid 0^{N+1}$
 - 7: $t6 \leftarrow t4 \mid 0^k$
 - 8: $t7 \leftarrow t6 - t5 + t4$
 - 9: $r \leftarrow a - t7$
 - 10: **if** $r \geq q$ **then**
 - 11: $r \leftarrow r - q$
 - 12: **end if**
 - 13: **return** r
-

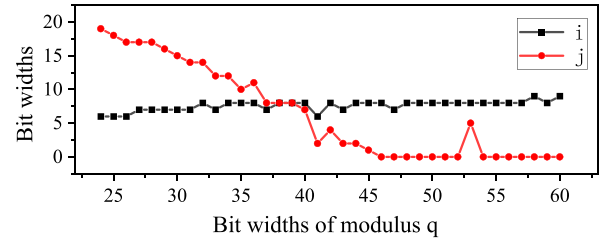


Fig. 5. Bit widths of effective values i for modulus q , and j for precomputed μ .

the bit width exceeds 26, there are more than 70 prime moduli. For commonly used bit widths, such as 32, 36, 54, 60, and 64, the number of moduli exceeds 200, which is sufficient to meet the requirements of FHE schemes.

We leverage this feature to extract the factor i as the valid bits of q and similarly process the precomputed value μ to obtain the factor j . The bit widths of i and j are smaller than those of q and μ , respectively. For i , its bit width depends on N , q , and the number of primes specified in the homomorphic parameter setup. Fig. 5 illustrates the bit width of i for a configuration that extracts 10 primes when $N = 16384$, showing that within the 24- to 60-bit range, the bit width of i remains between 6 and 9 bits. Furthermore, it demonstrates that the bit width of i does not significantly grow as the bit width of q increases.

On the other hand, the bit width of the pre-computed value j decreases as q increases and is significantly smaller than q in most cases. When q exceeds 46 bits, the value of j is almost zero, indicating that our approach achieves more substantial optimization for larger bit widths of q . Although the multiplication with μ is divided into two multiplications, the combined cost is lower than the original single multiplication due to the reduced bit widths. Moreover, since the pre-computed values

are replaced with i and j , this solution does not significantly increase the storage requirements. We represent the DSP resources utilized for modular reduction as follows, where $DSP_{\log i}$ denotes the DSP resources required for multiplication with a bit-width of i :

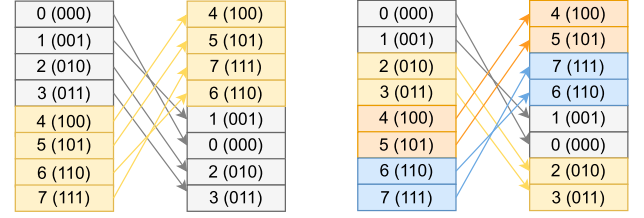
$$DSP_{modre}(\log_2 q) = 2DSP_{\log i} + DSP_{\log j} \quad (1)$$

The butterfly core of the NTT also includes a modular reduction, for which the existing work, HEAX [16] employs the Shoup algorithm. Although it reduces computational resources by decreasing the effective bit-width, the precomputed values double the number of twiddle factors. We replaced the Shoup algorithm with Algorithm 2, achieving the reduction in effective bit width without the need for additional precomputed values, offering a significant advantage for memory-constrained homomorphic encryption algorithms.

2) *Automorphism Acceleration*: Automorphism is a basic operation in rotation and should have comparable latency to other basic operations to maintain pipeline efficiency. Our work aims at the automorphism algorithm of the NTT-form version based on SEAL [22], shown in Algorithm 1. The NTT-form automorphism is performed after the NTT operation, where the NTT output is represented in point-value form in bit-reversed order. To facilitate computation, the input indices for this automorphism are renumbered, and the automorphism redefines the mapping based on the renumbered indices. For example, when $N = 8$, the data with bit-reversed indices (0, 4, 1, 5, 2, 6, 3, 7) is stored at sequential indices (0, 1, 2, 3, 4, 5, 6, 7) in an array. Consequently, Algorithm 1 shows that when the output data is accessed sequentially, the corresponding input data appears disordered. This makes it hard to retrieve several input data at once while the output data are partitioned and proceed in parallel. Additionally, our optimized architecture features configurable parallelism, enabling the adjustment of parallelism degrees in automorphism.

It processes the polynomial data after NTT operation and before INTT, where the polynomial is represented in point-value form in bit-reversed order. The NTT-form automorphism thus re-mapping the array data based on the reindexes the bit-reversed data; for example, when $N = 8$, the indices in bit-reversed order (0, 4, 1, 5, 2, 6, 3, 7) are stored sequentially in an array, on which the automorphism is applied.

To resolve the issue, we discovered a correspondence between the grouping of the input and output data. Fig. 6 illustrates the correspondence in array indices before and after the automorphism. We found that the high x bits of the output index are uniquely determined by the high x bits of the input index. Fig. 6(a) shows that for the most significant 1 bit, when the output index has a high 1 bit of “1”, the corresponding input index has a high 1 bit of “0”. Likewise, Fig. 6(b) reveals that for the highest 2 bits, when the output index is “00”, the input index is “10”. Similar patterns apply to other cases. As a result, we can partition the array into 2^x groups based on the x most significant bits. Each group comprises 2^y elements, where $y = \log_2 N - x$. By traversing the output index groups sequentially, we can determine the corresponding input index groups and extract all the elements in parallel from each group. Then, we put the elements directly into



(a) Divided into two groups, each processing four numbers in parallel. (b) Divided into four groups, each processing two numbers in parallel.

Fig. 6. Array index correspondence before and after automorphism shifting when $N = 8$.

Algorithm 3: Parallelism Automorphism in NTT-form.

Input: $a[N]$, $galois_elt$, parallelization of the low y bits;

Output: $\bar{a}[N]$

```

1: for  $i \leftarrow 0$  to  $2^y - 1$  do
2:    $rev\_i[i] \leftarrow reverse\_bits(i)$ 
3: end for
4: for  $j \leftarrow 0$  to  $\frac{N}{2^y} - 1$  do
5:    $rev\_j \leftarrow reverse\_bits(j)$ 
6:    $idx\_h\_t1 \leftarrow galois\_elt \cdot 2 \cdot rev\_j + galois\_elt - 1$ 
7:    $idx\_h\_t2 \leftarrow idx\_h\_t1 \gg (\log_2 N - y)$ 
8:    $idx\_h \leftarrow reverse\_bits(idx\_h\_t2)$ 
9:   for  $i \leftarrow 0$  to  $2^y - 1$  do ▷ Unroll the loop
10:     $rev\_l \leftarrow rev\_j | rev\_i[i]$  ▷ Concatenate two
      numbers
11:     $idx\_l\_t1 \leftarrow galois\_elt \cdot 2 \cdot rev\_l + galois\_elt - 1$ 
12:     $idx\_l\_t2 \leftarrow idx\_l\_t1 \gg (\log_2 N - y)$ 
13:     $idx\_l \leftarrow reverse\_bits(idx\_l\_t2)$ 
14:     $\bar{a}[j \cdot 2^y + i] \leftarrow a[idx\_h \cdot 2^y + idx\_l]$ 
15:   end for
16: end for
```

the sequential output index. Output data can be in burst-mode transmission, and input data requires an on-chip buffer. Furthermore, the parallelism can be adjusted by modifying the value of y . The algorithm shows in Algorithm 3.

C. Memory Management

1) *Data Layout of HP Ports*: In section III, we demonstrate that it becomes difficult to ensure high efficiency for ciphertexts with a varying number of RNS polynomials when we attempt to merge several RNS polynomials' data into a larger number to meet the HP port data bit-width.

Therefore, in addition to accessing the RNS polynomials in parallel, parallel access is also applied to the data within the polynomial. The challenge is to compute data within the RNS in parallel from off-chip memory while eliminating the on-chip buffer used to save the data off-chip. For most basic operations, it is easy to compute multiple elements in parallel internally, and NTT/INTT operations do not directly access external data, so they do not need to be considered. The hard part is the automorphism operation. Since we solved the issue by optimizing it

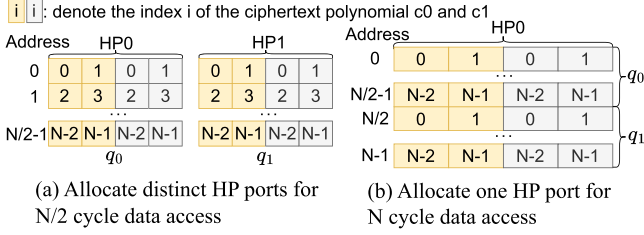


Fig. 7. Data concatenation scheme for HP ports.

in Section IV-B2, it can now be parallelized within groups, and the subscripts are continuous.

We splice the numbers of consecutive indices in RNS polynomial in Fig. 7. We can configure a data splicing scheme based on available HP ports and targeted transfer cycles for flexible design space exploration, prioritizing overall application performance. Separate HP ports are allocated to each RNS polynomial in Fig. 7(a). We adopt this approach to avoid wasting bandwidth when the basic function exclusively processes the q_0 polynomial. In cases where HP ports are limited, we employ time-sharing among different RNS polynomials. Fig. 7(b) shows the time-sharing of HP0 between q_0 and q_1 polynomials to enable N cycle read and write operations.

2) *Allocation of HP Ports*: In this section, we develop a model within our framework to estimate the BRAM/URAM or HP port requirements for buffers. The framework allows us to determine the optimal allocation for each buffer, whether placed on-chip or off-chip and adjust bandwidth or storage needs due to parameter variations. Specifically, we start with the solution from Section III-B that on-chip placement for NTT/INTT data and off-chip for other operations, then adjusting it according to memory constraints to find a balanced result.

To assess BRAM/URAM and HP port utilization, we establish the relationship between buffers, data bit width, and homomorphic operation parallelism. Let the parallelism of a basic operation op_i be denoted as P_{op_i} , where $op_i \in OP$, and OP represents the set of all basic operations. Buffers, holding coefficients from RNS polynomials, are denoted as $b_j \in B$, where B is the set of buffers used in homomorphic computations. If a buffer b_j is accessed by a basic operation op_i , it is represented as $\lambda_{j,i} = 1$; otherwise, $\lambda_{j,i} = 0$. We define Num_{b_j} as the number of digits from b_j that can be read in parallel per cycle, which depends on the parallelism of the homomorphic operation. The calculation of Num_{b_j} is detailed as follows:

$$Num_{b_j} = \max(\{P_{op_i} \mid op_i \in OP, \text{ and } \lambda_{j,i} = 1\}) \quad (2)$$

If Num_{b_j} is on-chip, we mark $M_{b_j} = 1$, else $M_{b_j} = 0$. We then specify the on-chip BRAM/URAM count needed for b_j as follows:

$$BRAM_{b_j}(N, \log_2 q, P_{op_i}) = \left\lceil \frac{N}{depth \cdot Num_{b_j}} \right\rceil \cdot \left\lceil \frac{Num_{b_j} \cdot \log_2 q}{width} \right\rceil \quad (3)$$

where the terms *depth* and *width* refer to the bit width and data depth of BRAM, and the same formula applies to URAM. For buffer b_j placed off-chip, we calculate the required HP ports. Since HP ports typically have a data width greater than ciphertext

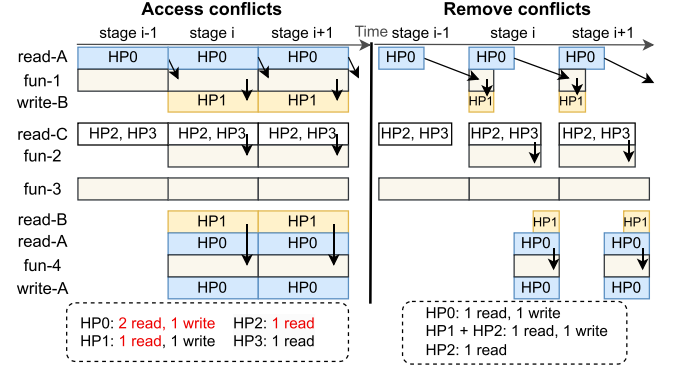


Fig. 8. An example of data access conflicts at HP ports and the solution.

numbers, the packing capacity per HP port, denoted as hp_{pack} , is calculated as follows:

$$hp_{pack} = \left\lfloor \frac{hp_{width}}{\log_2 q} \right\rfloor \quad (4)$$

where hp_{width} and $\log_2 q$ denote the maximum data width of the HP port and the bit width of q . The following formula expresses the number of HP ports used by off-chip buffer b_j .

$$\tilde{hp}_{b_j} = \left\lceil \frac{Num_{b_j}}{hp_{pack}} \right\rceil \quad (5)$$

3) *Avoiding Data Access Conflicts*: During the pipeline stage of homomorphic operations, multiple accesses to the HP ports may occur due to repeated visits to buffers, causing conflicts by exceeding the port's access limit. Fig. 8 depicts the computation process of rotation, where at pipeline stage " i ", multiple basic functions run in parallel, necessitating two reads and one write to HP0. Furthermore, since HP1 and HP2 share a DDR controller according to the FPGA architecture of ZCU 102, the controller also requires two reads. These may cause access conflicts, making each stage take more cycles than expected or delaying some functions to the next stage. To tackle this problem, we've shortened each HP port's usage time to stagger accesses within the same pipeline stage. Fig. 8(b) shows adjustments to the cycles of function-1,2,4, minimizing each port's access time to prevent conflicts within a stage.

We develop a model for potential access conflicts and solutions, leveraging DDR's dual-channel read-write capability to define HP ports read and write separately. If op_i reads b_j data, we mark $r_{j,i} = 1$; if not, $r_{j,i} = 0$. Similarly, if op_i writes to b_j , $w_{j,i} = 1$; otherwise, $w_{j,i} = 0$. We denote reads via HP port as $hp_r_{b_j}$ and actual HP usage post-conflict resolution as $hp_r_{b_j}$, resulting in the formula:

$$hp_r_{b_j} = \begin{cases} \tilde{hp_r_{b_j}} \cdot \sum_{r_{j,i}=1} Lat_{op_i} \leq Lat_{up} \\ \tilde{hp_r_{b_j}} \cdot \sum_{r_{j,i}=1} r_{j,i} \\ \sum_{r_{j,i}=1} Lat_{op_i} > Lat_{up} \end{cases} \quad \text{where } M_{b_j} = 0 \quad (6)$$

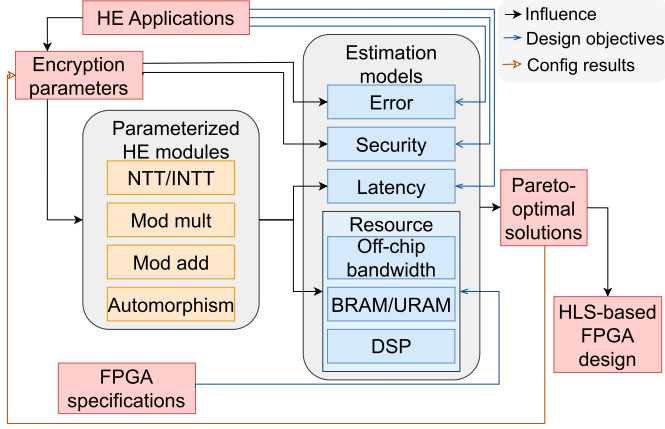


Fig. 9. Framework of design space exploration.

where, Lat_{op_i} denotes the latency of the operation, and is influenced by the parameters N and P_{op_i} .

$$Lat_{op_i}(N, \log_2 q, P_{op_i}) = \begin{cases} \frac{N}{P_{op_i}}, & op_i \in OP \text{ but not NTT} \\ \frac{N \cdot \log_2 N}{2 \cdot P_{op_i}}, & op_i \in OP \text{ and is NTT} \end{cases} \quad (7)$$

Furthermore, Lat_{up} is an upper bound on the latency of each operation.

Considering the mapping of 4 HP ports to 3 DDR controllers, where HP1 and HP2 share one DDR controller for data scheduling, we treat HP1 and HP2 as one port for special handling. The buffer set for these two HP ports is denoted as B' . $hp_r_{b_k}$ formula:

$$hp_r_{b_k} = \begin{cases} \max\{\tilde{hp_r_{b_k}}, \sum_{r_{k,i}=1} Lat_{op_i} \leq Lat_{up}\} \\ \max\{\tilde{hp_r_{b_k}}, \sum_{r_{k,i}=1} r_{k,i}\} \\ , \sum_{r_{k,i}=1} Lat_{op_i} > Lat_{up} \end{cases} \quad \text{where } b_k \in B', M_{b_j} = 0 \quad (8)$$

Similarly, the formula applies to writes to HP ports. We calculate the number of HP ports used in each stage during the whole computation, and since each stage's use is independent, we total the HP ports used, expressed as:

$$HP_{sum}(N, \log_2 q, P_{op_i}) = \max_{stage} \left\{ \max \left(\sum_{b_j \in stage_t} hp_r_{b_j}, \sum_{b_j \in stage_t} hp_w_{b_j} \right) \right\} \quad (9)$$

V. DESIGN SPACE EXPLORATION

We introduce a framework for homomorphic applications on resource-constrained FPGAs, offering a multi-objective optimization design focusing on error, security, and latency. The design process is detailed in Fig. 9. Within this framework, parameterized HE operation modules are developed using Vitis

HLS [37]. The modules integrate our optimization strategies for basic homomorphic operations to balance performance and resource utilization across various HE module parallelisms and parameters. Next, we integrate existing research to model the error and security affected by homomorphic application parameters. We also develop comprehensive models for the application's latency and resource consumption, taking into account its HE operation usage patterns. The resource models include our data allocation optimization strategy to balance on-chip memory and off-chip bandwidth. Finally, we achieve Pareto-optimal outcomes for multi-objective optimization by repeatedly iterating over homomorphic parameters within resource constraints.

Parameter generation: The parameters generated by our framework include N and a set of small moduli q . In the resource and latency formulas, $\log_2 q$ is used as the bit width of q for DSE. Once $\log_2 q$ is generated, the corresponding q values are further derived for error and security computations based on [38], [39]. The variables N and $\log_2 q$ are used in Equations (1) (3) (7) (9) to estimate DSP, BRAM, latency and HP port utilization. After completing the DSE process, the framework provides the optimized parameter values for implementation.

Multi-objective problem: The multi-objective problem is formulated as:

$$\text{Minimize } Lat, Err; \quad \text{Maximize } Sec \quad (10)$$

where Lat , Err , and Sec represent latency, error, and security level, respectively. Latency is computed based on the latency of each operation and the pipeline interval count, denoted as $countI_{op_i}$, and is defined as follows:

$$Lat = \sum_{op_i \in OP} countI_{op_i} \cdot Lat_{op_i} \quad (11)$$

The security level λ is expressed as $\lambda \xrightarrow{G} (N, Q_l)$, where G represents the relationship between the parameters N, Q_l and the security level [38]. Thus, the security level is defined as:

$$Sec = G^{-1}(N, Q_l) \quad (12)$$

For the error formula, d denotes the depth of multiplication, and ϵ_i satisfies $\frac{\Delta}{q_i} \in (1 - 2^{-\epsilon_i}, 1 + 2^{-\epsilon_i})$, where q_i denotes the i index of the RNS moduli, and Δ is the scale factor. The error formula is defined as:

$$Err = \sum_{k=1}^d 2^{-\epsilon_{l-k+1} + (d-k)} \quad (13)$$

Next, we present the resource constraints for the FPGA, defined as follows:

$$DSP_{sum} = \sum_{op_i \in OP} count_{op_i} \cdot DSP_{op_i} \leq DSP_{max}$$

$$BRAM_{sum} = \sum_{b_j \in B, M_{b_j}=1} BRAM_{b_j} \leq BRAM_{max}$$

$$HP_{sum} \leq HP_{max} \quad (14)$$

where DSP_{max} , $BRAM_{max}$, and HP_{max} represent the maximum DSP, BRAM, and HP port capacities of the FPGA. $count_{op_i}$ denotes the number of basic operations op_i .

TABLE II
FPGA PLATFORMS USED IN THE EVALUATION

Board	On-Chip Resources				DRAM
	DSP	LUT	BRAM	URAM	Bandwidth (GB/s)
ZCU102	2520	274K	912	-	19.2 (DDR)
ZCU104	1728	230K	312	96	19.2 (DDR)
U280	9024	1304K	2016	960	38(DDR)+460(HBM)

The design solution encompasses both parameter settings and hardware configuration. The parameter results include N and Q_L , and the bit width of q for small prime moduli. The design space includes value ranges for N between 2^{12} to 2^{16} , and q from 20 to 60 bits, constituting hundreds of combinations. Meanwhile, the results of the hardware configuration contain the parallelism P_{opi} for basic HE operation and the allocation of buffer b_j , covering the usage of BRAM/URAM and the HP port allocations. For a given parameter set, thousands of hardware configurations are possible. We initially identify configurations using an exhaustive search, which only takes seconds. Next, by iteratively tweaking the parameters, we reduce the search scope. For example, if there is no solution for a parameter such as “ $N = 2^{14}$, $q = 54$ ”, performance with smaller parameters is tested in the next iteration. Overall, the time overhead of DSE is manageable compared to FPGA programming and bitstream generation.

VI. EXPERIMENTS

A. Experimental Setup

We evaluate DAHE with two different categories of FPGA platforms: embedded development boards and high-performance FPGA accelerator cards. These boards differ significantly in terms of computation resources and memory hierarchy. The adopted FPGA platforms are detailed in Table II. We adopt the high-level synthesis (HLS) tool chain to implement DAHE for deployment on different FPGA platforms. In specific, we use the Vitis 2023.1 for RTL code generation from HLS and compile RTL code into bitstream executables.

Benchmarks: We use the following applications as benchmarks to demonstrate DAHE’s flexibility and performance benefits.

HEXL-benchmarks. HEXL [25] is a widely recognized homomorphic encryption acceleration framework for CPU platforms. It integrates with leading homomorphic encryption libraries, such as Microsoft SEAL [22]. The HEXL benchmarks cover diverse homomorphic matrix multiplications with various packing methods, operation types (plaintext-ciphertext or ciphertext-only), and matrix sizes.

Neural networks. We selected LoLa [40] for MNIST and CIFAR as the benchmark for homomorphic neural networks, denoted by LoLa-MNIST and LoLa-CIFAR, respectively. Both networks consist 5-layers. However, LoLa-CIFAR contains about one hundred times more homomorphic operations than LoLa-MNIST. Such an increase imposes significant challenges for managing data allocation on the on-chip and off-chip memory.

TABLE III
COMPARISON OF THROUGHPUT (OP/S) FOR HE OPERATIONS USING PARAMETER SETS A AND B ACROSS HEAX [16], FAB [6] AND MEDHA [11]

	Work	Device				
			NTT	Mult	Rotation	Rescale
A	HEAX	Stratix 10	42K (2.97 ×)	2.6K (1.38 ×)	/	/
	Medha	U250	27.8K (4.45 ×)	2.0K (1.8 ×)	/	5.8K (2.36 ×)
	FAB	U280	167K (1.5 ×)	5.7K (1.26 ×)	6.3K (1.16 ×)	51.3K (0.95 ×)
	DAHE	Stratix 10	125K	3.6K	3.7K	13.7K
	DAHE	U250	125K	3.6K	3.7K	13.7K
	DAHE	U280	251K	7.2K	7.3K	48.8K
B	HEAX	Arria 10	89.5K (1.63 ×)	44.7K (1.09 ×)	/	/
	DAHE	Arria 10	146.4K	48.8K	48.8K	73.2K
	DAHE	ZCU102	146.4K	48.8K	48.8K	73.2K
	DAHE	ZCU104	73.2K	24.4K	24.4K	36.6K
C	FAB	U280	/	584 (1.19 ×)	636 (1.19 ×)	5.2K (0.83 ×)
	DAHE	U280	54.9K	653	763	4.3K

Note: For Set-A, $N = 2^{14}$, $\log Q = 438$ and $L = 8$. For Set-B, $N = 2^{12}$, $\log Q = 108$ and $L = 2$. For Set-C, $N = 2^{16}$, $\log Q = 1242$ and $L = 23$.

*Mult includes CMult and Relinearization.

B. HE Operations Performance

High-level operations: We evaluate the performance of homomorphic operations and compare DAHE with state-of-the-art FPGA-based homomorphic accelerators, including HEAX [16], FAB [6], and Medha [11], as summarized in Table III. The evaluation includes operations such as NTT, ciphertext multiplication (CMult and relinearization), rotation, and rescale. All of these operations involve modular reduction, while automorphisms are specific to rotation. For a fair comparison, we adopt the same homomorphic encryption parameters and FPGA platforms, including Xilinx U280, U250, Intel Arria 10, and Stratix 10, which vary in resource capacity and off-chip memory bandwidth. Some literature studies have built on Intel Altera FPGA devices, which support the Intel HLS Compiler. In order to provide a tentative comparison, we impose hardware resource constraints on the Xilinx U280 device to imitate the resource availability of corresponding Altera FPGA devices.

The results demonstrate that DAHE achieves a 1.16x to 4.45x performance improvement for most HE operations across various FPGAs. Compared to HEAX, this improvement is primarily attributed to reduced DSP usage for modular multiplication within a single NTT core, allowing more NTT cores to be placed on devices like the Stratix 10 and Arria 10 FPGAs. For example, the number of NTT cores in the Arria 10 increased from 8 to 16 compared to HEAX. Furthermore, while Arria 10 offers more resources than ZCU102, and ZCU102 has more resources than ZCU104, the performance of Arria 10 is only comparable to ZCU102 and outperforms ZCU104. This is because the additional resources on Arria 10 are insufficient to instantiate enough FHE basic units to effectively reduce the pipeline interval between operations. In contrast, ZCU104’s limited resources prevent it from being configured similarly to ZCU102, further constraining its performance.

Compared to FAB, DAHE achieves better performance by deploying 326 modular multiplication units, 256 of which are dedicated to NTT computations, whereas FAB uses only 256 modular multiplication units in total. Similarly, DAHE outperforms Medha in terms of performance on the U250 FPGA. While the U250 offers more on-chip resources than the U280—such as DSPs, BRAM, and LUTs—allowing for the placement

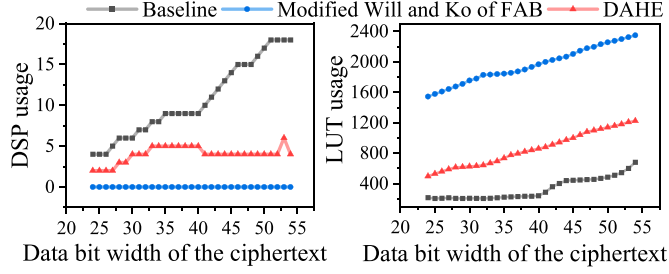


Fig. 10. DSP and LUT usage of modular reduction compared to baseline and FAB [6].

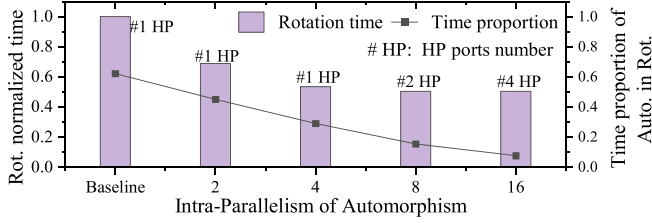


Fig. 11. Performance of automorphism.

of additional computational units, its off-chip memory bandwidth is limited to 77 GB/s compared to the U280's 460 GB/s HBM bandwidth. This bandwidth constraint limits the data supply required for efficient computation, thereby impacting the U250's performance.

Modular reduction: To evaluate the effectiveness of our modular reduction optimization, we compared it with two methods: the conventional Barrett reduction (set as the baseline) and the modified Will and Ko's modular reduction proposed by FAB. Both methods were implemented using HLS across varying data bit widths. Since FAB's algorithm supports variable bit widths while its original hardware optimization targets only 54-bit inputs, we focus on FAB's algorithm to handle a broader range of data widths. Fig. 10 compares the DSP and LUT resource usage of the three methods. Mathematically, the FAB algorithm requires 20 additions/subtractions and 9 shifts, while our method reduces these to 7 and 6, respectively, as shown in Algorithm 2, thereby decreasing LUT usage. Additionally, we replace the original multiplication with a shortened bit width parameter compared to the baseline, further reducing DSP usage. The experimental results in Fig. 10 show that our method achieves a balanced utilization of the DSP and LUT resources, with both usages between the two methods. Besides, DSP usage does not increase when bit widths exceed 42 bits due to the reduction in pre-computed values in Fig. 5. Therefore, our method is particularly advantageous for larger data bit width computations.

Automorphism: Fig. 11 demonstrates the influence of automorphism optimization on Rotation performance. The optimization strategy of DAHE facilitates the parallel handling of multiple sets of number mappings within the automorphism. Thus, we consider a non-parallel automorphism (i.e., with a parallelism of "1") as the baseline. The efficiency of the improved Rotation is $1.98 \times$ higher than the baseline. As the parallelism degree rises, the cycle count for automorphism reduces.

TABLE IV
LATENCY AND PERFORMANCE-PER-WATT (PPW) COMPARISON WITH REPORTED FPGA IMPLEMENTATION OF FxHENN ON ZCU102 [33]

		FxHENN [33]- ZCU102	DAHE- ZCU102	Yang [15]- U280	DAHE- U280
LoLa-MNIST	Lat. (s)	0.24	0.19 (1.26 $\times$)	0.02	0.0037 (5.4 $\times$)
	PPW	0.42	0.48 (1.15 $\times$)	-	3.22
LoLa-CIFAR	Lat. (s)	254	41.75 (6.09 $\times$)	7.2	1.24 (5.8 $\times$)
	PPW	0.0004	0.002 (5.53 $\times$)	-	0.0096

However, the performance plateaus beyond the parallelism of "8" for Rotation. This is because, at lower levels of parallelism, the cycle count for automorphism surpasses that of other basic operations, making it the bottleneck. When parallelism surpasses "8", the cycle count for automorphism drops, causing the duration of other basic operations to become the new bottleneck. Thus, only increasing automorphism's parallelism does not further decrease the total time for Rotation. Furthermore, increasing automorphism's parallelism heightens the usage of HP ports for off-chip data access, limiting the number of available ports for other basic operations and affecting overall parallelism. This experiment emphasizes the importance of meticulously setting the parallelism for basic operations through design space exploration to enhance performance and FPGA resource utilization. It is essential to avoid resource wastage due to excessive parallelism and to prevent bottlenecks from insufficient parallelism.

C. HE Applications Performance

LoLa HE-CNN: We implemented plaintext-ciphertext inference for the LoLa-MNIST and CIFAR homomorphic neural networks on the Xilinx ZCU102 and U280 platforms. Table IV presents performance comparisons with state-of-the-art FPGA-based solutions, FxHENN [33] and [15], using the same FHE parameters and FPGA boards. On the ZCU102, DAHE outperformed FxHENN with a $1.26 \times$ improvement for the MNIST network and a $6.09 \times$ improvement for the CIFAR network. FxHENN's DSE framework does not account for off-chip memory bandwidth, leading to underutilized memory resources. The issue is significant for the CIFAR network, which involves larger data volumes than the MNIST network (2471 MB for CIFAR and 15 MB for MNIST) [33].

On the U280, DAHE achieves $5.4 \times$ and $5.8 \times$ performance improvements over Yang [15] for the two HE-NNs, respectively. This improvement is primarily attributed to the U280's high off-chip memory bandwidth (460 GB/s), which necessitates sufficient computational units to process the data efficiently. Our modular reduction optimization reduces DSP usage, enabling the configuration of more modular multiplication units to fully utilize the data fetched from HBM. Specifically, we configure 288 NTT cores, 288 INTT cores, and 96 modular multiplication units, utilizing 6048 DSPs (67% of the board's capacity) and 73% of the total on-chip memory. The DSE framework efficiently allocates 28 out of 32 HBM ports, optimizing computational and memory resource utilization to improve performance.

In this work, energy efficiency is formally defined as $(1/\text{Latency})/\text{Power}$. The ZCU102 implementation consumes

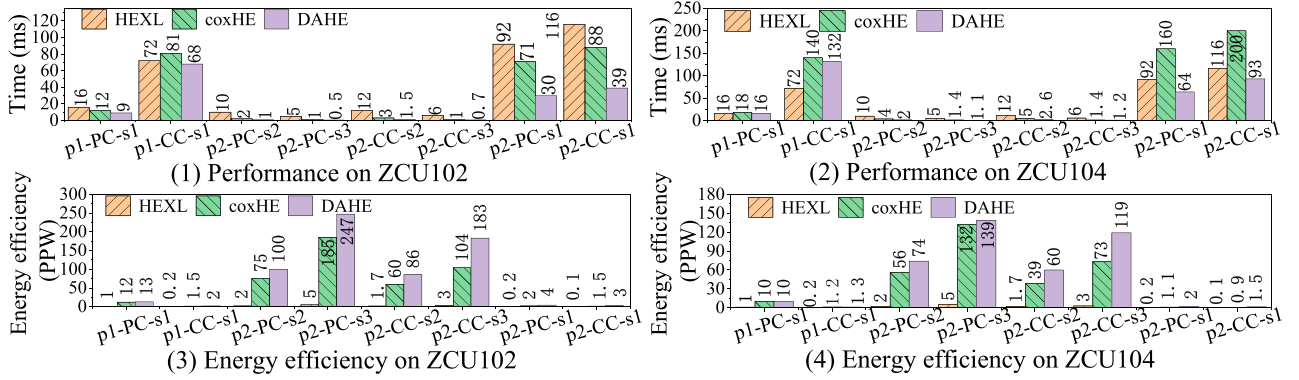


Fig. 12. Performance and energy efficiency of HEXL benchmarks, compared to HEXL [25] and coxHE [32].

approximately 11W, which is comparable to FxHENN. It achieves $1.15\times$ and $5.53\times$ improvements in energy efficiency for the MNIST and CIFAR networks, respectively. The U280 implementation consumes 84.3W but delivers significantly higher computational performance. Although it consumes more power, the U280 achieves better energy efficiency than the ZCU102 due to its significant performance gains.

HEXL benchmarks: We evaluated HEXL benchmarks and compared performance with CPU implementations of HEXL [25] and the advanced FPGA solution coxHE [32], using results derived from coxHE [32]. The CPU-based benchmarks are conducted on an Intel Core i7-8700 with a six-core, twelve-thread, operating at 3.70 GHz with a 95W TDP and a measured runtime power between 50-90W. We perform DAHE on the Xilinx ZCU102 and ZCU104 platforms, the same as coxHE. Fig. 12 presents comparison results, focusing on matrix-vector multiplication of HEXL benchmarks with varying sizes, packing methods, and operation types. Specifically, for the name of each benchmark, “p1” for single element packing, “p2” for multiple elements packing; “PC” for plain-cipher operations, “CC” for cipher-cipher operations; sizes for matrix multiplications are s1: $(100\times 10)\times(10\times 1)$, s2: $(1\times 2048)\times(2048\times 1)$, s3: $(1\times 10)\times(10\times 1)$.

For latency, DAHE outperforms the multithreaded HEXL in most benchmarks, achieving performance improvements by $4\times$ to $9\times$ and offering performance enhancements over coxHE from $1.09\times$ to $2.51\times$. Notable improvements are observed in benchmarks “p2-PC-s1” and “p2-CC-s1”, due to significant improvements in memory management and optimized effect for Rotation, which are critical given the benchmarks’ requirement for 400 rotations due to specific matrix sizes and packing methods. Furthermore, the ZCU102 platform demonstrates superior performance compared to the ZCU104, mainly due to its higher number of DSPs and BRAMs, which doubles the parallelism in basic operations. Regarding power efficiency, both boards outperform the CPU works, with improvements ranging from $5\times$ to $59\times$. For coxHE, the energy efficiency on the ZCU102 is improved by $1.13\times$ to $2.19\times$, and on the ZCU104, the improvement reaches up to $2.04\times$. This experiment demonstrates the DAHE framework’s adaptability across various homomorphic applications and hardware configurations, highlighting its performance and energy efficiency benefits.

TABLE V
PERFORMANCE COMPARISON OF COXHE [32] DSE CONFIGURATIONS, WITH “PORTS” DENOTING THE NUMBER OF HP/HBM PORTS UTILIZED

	N	$\log q$	Sec.	Error	Scheme	Ports	Lat. (ms)
S1	8192	33	192	9.53 E-06	coxHE-104	-	34.56
					DAHE-104	4	13.47
					DAHE-u280	23	2.4
S2	8192	37	192	9.53 E-07	coxHE-104	-	67.34
					DAHE-104	4	26.9
					DAHE-u280	23	3.36

D. Design Space Exploration

Our framework automatically generates the Pareto optimal solutions for a given software kernel and target FPGA device. Compared to coxHE [32], the main improvements of our framework are: 1) An added optimization factor for off-chip bandwidth utilization in the model, resulting in DSE outcomes that outperform coxHE. Table V shows a $2.56\times$ improvement in latency under S2 case; 2) Our framework incorporates modeling for HBM to fully leverage the advantages of high-performance FPGA accelerators. In Table VI, DAHE is capable of exploring the design space for the U280 FPGA hardware design and supports large parameters (e.g., $N = 2^{16}$). The design space exploration includes configurations for the parallelism of basic operations, denoted as P_{op_1} , where P_{op_0} , P_{op_1} , and P_{op_2} represent the parallelism for NTT and two modular computations, respectively. The results include resource usage metrics such as DSPs, LUTs, FFs, BRAMs, and URAMs. Compared to FAB, the LUT, BRAM, and URAM usage are lower for the U280 configuration with $\log q = 54$, while DSP usage remains similar. Our framework offers a broader design space in both homomorphic parameters and hardware resources, along with more potential configuration points for performance enhancement, making it a more comprehensive DSE framework.

E. Performance of Memory Management

This section focuses on experiments for optimizing off-chip memory bandwidth. Our methods include placing immediate data buffers in off-chip memory and optimizing HP port allocation to avoid access conflicts. We compare two baselines with DAHE: Baseline-1 places all buffers on-chip, which exhausts

TABLE VI
CONFIGURATION AND PERFORMANCE OF OPTIMAL CHOICES ON ZCU104, ZCU102 AND U280

Device	Homomorphic Parameters		Hardware Configuration				Utilization(%)					Performance		
	N	$\log q$	P_{op0}	P_{op1}	P_{op2}	HP	DSP	LUT	FF	BRAM	URAM	Err.	Sec.	Lat.(ms)
ZCU104	4096	27	4	1	8	4	20.7	47.68	19.75	78.52	66.67	0.000305	128	6.243
	8192	29	4	1	8	4	41.4	50.13	20.65	78.52	66.67	0.000336	256	13.447
ZCU102	4096	27	8	1	16	4	26.19	56.29	22.58	73.68	-	0.000305	128	3.842
	8192	30	8	1	16	4	52.38	65.98	26.39	73.68	-	4.58E-05	192	7.657
	16384	36	8	1	8	4	44.76	80.9	31.8	73.68	-	1.53E-05	256	15.368
U280	4096	27	32	32	64	31	45.45	55.31	31.08	69.84	46.67	0.000305	128	0.78
	8192	54	16	16	32	31	54.68	58.22	36.96	75.30	31.46	6.00E-11	128	3.381
	16384	54	16	16	32	31	54.68	58.22	36.96	75.30	31.46	6.00E-11	256	7.204
	32768	54	16	16	32	31	54.68	58.22	36.96	75.30	31.46	1.53E-10	256	15.368
	65536	27	32	32	64	31	45.45	55.31	31.08	69.84	46.67	0.002686	256	16.329

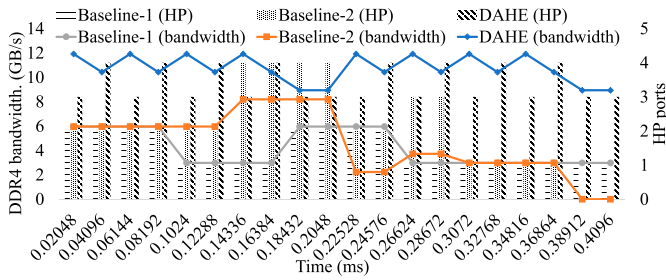


Fig. 13. Bandwidth of rotation & HAdd within the pipeline.

TABLE VII
PERFORMANCE COMPARISON

	P_{NTT}	On-/Off-Chip Buffers (%)	BRAM18K	DDR (MB)	HP Ports	Perf. (op/s)
Baseline-1	4	100/0	992 (55%)	0	2	2441
Baseline-2	8	66/34	1298 (71%)	1.4	4	2380
DAHE	8	66/34	1298 (71%)	1.4	4	4761

BRAM resources, forcing a reduction in computational units to meet BRAM bandwidth limitations; Baseline-2 places some buffers off-chip but does not optimize HP ports allocation. Fig. 13 shows off-chip bandwidth and HP port usage for homomorphic matrix multiplication, while Table VII presents memory usage and throughput for rotation and addition operations, the main components of homomorphic matrix multiplication.

In Baseline-1, computational units are reduced, using 55% of BRAM resources (or 110% for parallelism of 8), resulting in low off-chip bandwidth usage (under 6 GB/s) and a maximum of 2 HP ports. Baseline-2 increases off-chip buffer usage and utilizes all 4 HP ports during 0.14–0.2 ms. However, it fails to allocate additional HP ports for subsequent computations phases, leading to pipeline stalls. DAHE addresses this by rearranging HP port usage between 0.02–0.16 ms to prefetch data for the next computation phase, improving off-chip bandwidth utilization and eliminating pipeline bubbles.

VII. CONCLUSION AND FUTURE WORK

In this paper, we propose an FPGA acceleration framework for fully-fledged CKKS based FHE kernels with HLS design flow. We initiated the optimization design for FHE operation modules, such as Automorphism and modular reduction architecture. Our proposed memory management scheme alleviates the problem of on-chip memory shortage and improves the utilization of off-chip memory bandwidth, which has a significant impact on performance. The overall framework automatically explores the design space including encryption parameters and hardware configurations, and produce Pareto optimal solutions that balance security, performance, and approximation error.

This work aims to optimize memory usage for FHE computing, with a primary focus on alleviating the limitations of on-chip BRAM and efficiently utilizing off-chip bandwidth. Future work will explore memory optimization for FHE applications with larger parameters and greater data volumes, such as bootstrapping, HELR, and ResNet20. These scenarios can involve data sizes ranging from several gigabytes to tens of gigabytes—for instance, evaluation keys and plaintext matrices for bootstrapping alone can require up to 8 GB [2]. Such large-scale data exceeds the capacity of HBM on U280-like FPGAs, leading to performance degradation due to frequent data transfers. To address this, future research will investigate data placement strategies for HBM and explore the use of DDR memory, which offers greater capacity but lower bandwidth when integrated into U280-like FPGAs.

REFERENCES

- [1] N. Samardzic et al., “F1: A fast and programmable accelerator for fully homomorphic encryption,” in *Proc. MICRO: 54th Annu. IEEE/ACM Int. Symp. Microarchit.*, Virtual Event, Greece, Oct. 18–22. New York, NY, USA: ACM, 2021, pp. 238–252.
- [2] J. Kim et al., “ARK: Fully homomorphic encryption accelerator with runtime data generation and inter-operation key reuse,” in *Proc. 55th IEEE/ACM Int. Symp. Microarchit., MICRO*, Chicago, IL, USA, Oct. 1–5. Piscataway, NJ, USA: IEEE, 2022, pp. 1237–1254.
- [3] R. Gilad-Bachrach et al., “Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy,” in *Proc. ICML*, USA, vol. 48. JMLR.org, 2016, pp. 201–210.

- [4] S. Kim et al., "BTS: An accelerator for bootstrappable fully homomorphic encryption," in *Proc. ISCA: 49th Annu. Int. Symp. Comput. Archit.*, New York, NY, USA, Jun. 18–22. New York, NJ, USA: ACM, 2022, pp. 711–725.
- [5] J. Kim, S. Kim, J. Choi, J. Park, D. Kim, and J. H. Ahn, "SHARP: A short-word hierarchical accelerator for robust and practical fully homomorphic encryption," in *Proc. 50th Annu. Int. Symp. Comput. Archit.*, Orlando, FL, USA, Jun. 17–21. New York, NJ, USA: ACM, 2023, pp. 18:1–18:15.
- [6] R. Agrawal et al., "FAB: An FPGA-based accelerator for bootstrappable fully homomorphic encryption," in *Proc. IEEE Int. Symp. High-Performance Comput. Archit., HPCA*, Montreal, QC, Canada, Feb. 25–Mar. 1. Piscataway, NJ, USA: IEEE, 2023, pp. 882–895.
- [7] Y. Yang, H. Zhang, S. Fan, H. Lu, M. Zhang, and X. Li, "Poseidon: Practical homomorphic encryption accelerator," in *Proc. IEEE Int. Symp. High-Performance Comput. Archit., HPCA*, Montreal, QC, Canada, Feb. 25–Mar. 1. Piscataway, NJ, USA: IEEE, 2023, pp. 870–881.
- [8] S. Fan, Z. Wang, W. Xu, R. Hou, D. Meng, and M. Zhang, "TensorFHE: Achieving practical computation on encrypted data using GPGPU," in *Proc. IEEE Int. Symp. High-Performance Comput. Archit., HPCA*, Montreal, QC, Canada, Feb. 25–Mar. 1. Piscataway, NJ, USA: IEEE, 2023, pp. 922–934.
- [9] E. Kirshanova, C. Marcolla, and S. Rovira, "Guidance for efficient selection of secure parameters for fully homomorphic encryption," *Cryptol. ePrint Arch.*, Paper 2024/1001, 2024.
- [10] Q. Lou, S. Bian, and L. Jiang, "Autoprivacy: Automated layer-wise parameter selection for secure neural network inference," in *Proc. NeurIPS*, Dec. 6–12, 2020, Virtual, 2020.
- [11] A. C. Mert, S. Kwon, Y. Shin, D. Yoo, Y. Lee, and S. S. Roy, "Medha: Microcoded hardware accelerator for computing on encrypted data," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2023, no. 1, pp. 463–500, 2023.
- [12] J. H. Ju et al., "Neujeans: Private neural network inference with joint optimization of convolution and FHE bootstrapping," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security, CCS*, Salt Lake City, UT, USA, Oct. 14–18, B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie, Eds., New York, NJ, USA: ACM, 2024, pp. 4361–4375.
- [13] E. Lee et al., "Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions," in *Proc. Int. Conf. Mach. Learn., ICML*, Jul. 17–23, 2022, Baltimore, MD, USA, in *Proceedings of Machine Learning Research*, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvári, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 2022, pp. 12403–12422.
- [14] X. Ren et al., "HEDA: Multi-attribute unbounded aggregation over homomorphically encrypted database," *Proc. VLDB Endow.*, vol. 16, no. 4, pp. 601–614, 2022.
- [15] Y. Yang, R. Kannan, and V. K. Prasanna, "A framework for generating accelerators for homomorphic encryption operations on FPGAs," in *Proc. 35th IEEE Int. Conf. Appl.-Specific Syst., Archit. Processors, ASAP*, Hong Kong, Jul. 24–26, 2024. Piscataway, NJ, USA: IEEE, 2024, pp. 61–70.
- [16] M. S. Riaz, K. Laine, B. Pelton, and W. Dai, "HEAX: An architecture for computing on encrypted data," in *Proc. ASPLOS, Switzerland*, New York, NJ, USA: ACM, 2020, pp. 1295–1309.
- [17] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(Leveled) Fully homomorphic encryption without bootstrapping," in *Innovations in Theoretical Computer Science, USA*. New York, NJ, USA: ACM, 2012, pp. 309–325.
- [18] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," *IACR Cryptol. ePrint Arch.*, 2012, p. 144.
- [19] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds," in *Proc. Asiacrypt.*, vol. 10031, 2016, pp. 3–33.
- [20] J. H. Cheon, A. Kim, M. Kim, and Y. S. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. ASIACRYPT*, Hong Kong, China, vol. 10624. Cham, Switzerland: Springer International Publishing, 2017, pp. 409–437.
- [21] S. Halevi and V. Shoup, "Design and implementation of HELib: A homomorphic encryption library," *IACR Cryptol. ePrint Arch.*, 2020, p. 1481.
- [22] "Microsoft SEAL (release 4.1)," Microsoft Res., Redmond, WA, USA, Jan. 2023. [Online]. Available: <https://github.com/Microsoft/SEAL>
- [23] P. Barrett, "Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor," in *Proc. CRYPTO*, Santa Barbara, CA, USA, 1986, pp. 311–323.
- [24] P. L. Montgomery, "Modular multiplication without trial division," *Math. Comput.*, vol. 44, pp. 519–521, 1985.
- [25] F. Boemer, S. Kim, G. Seifu, F. D. M. de Souza, and V. Gopal, "Intel HEXL: Accelerating homomorphic encryption with Intel AVX512-IFMA52," 2021, *arXiv:abs/2103.16400*.
- [26] W. Jung, S. Kim, J. H. Ahn, J. H. Cheon, and Y. Lee, "Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with gpus," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2021, no. 4, pp. 114–148, 2021.
- [27] N. Samardzic et al., "Craterlake: A hardware accelerator for efficient unbounded computation on encrypted data," in *Proc. ISCA*. New York, NJ, USA: ACM, 2022, pp. 173–187.
- [28] M. Kim, D. Lee, J. Seo, and Y. Song, "Accelerating HE operations from key decomposition technique," in *Proc. CRYPTO*, Santa Barbara, CA, USA, Aug. 20–24, Part IV, vol. 14084. Berlin, Heidelberg: Springer-Verlag, 2023, pp. 70–92.
- [29] A. Mukherjee et al., "ModHE: Modular homomorphic encryption using module lattices: Potentials and limitations," *Cryptol. ePrint Arch.*, Paper 2023/895, 2023.
- [30] D. B. Cousins, K. Rohloff, and D. Sumorok, "Designing an FPGA-accelerated homomorphic encryption co-processor," *IEEE Trans. Emerg. Top. Comput.*, vol. 5, no. 2, pp. 193–206, Apr./Jun. 2017.
- [31] S. S. Roy, F. Turan, K. Järvinen, F. Vercauteren, and I. Verbauwhede, "FPGA-based high-performance parallel architecture for homomorphic computing on encrypted data," in *Proc. 25th IEEE Int. Symp. High Performance Comput. Archit., HPCA*, Washington, DC, USA, Feb. 16–20. Piscataway, NJ, USA: IEEE, 2019, pp. 387–398.
- [32] M. Han, Y. Zhu, Q. Lou, Z. Zhou, S. Guo, and L. Ju, "coxHE: A software-hardware co-design framework for FPGA acceleration of homomorphic computation," in *Proc. Design, Automat. Test Europe Conf. Exhibit., DATE*, Antwerp, Belgium, Mar. 14–23. Piscataway, NJ, USA: IEEE, 2022, pp. 1353–1358.
- [33] Y. Zhu, X. Wang, L. Ju, and S. Guo, "FxHENN: FPGA-based acceleration framework for homomorphic encrypted CNN inference," in *Proc. IEEE Int. Symp. High-Performance Comput. Archit., HPCA*, Montreal, QC, Canada, Feb. 25–Mar. 1. Piscataway, NJ, USA: IEEE, 2023, pp. 896–907.
- [34] R. Agrawal, L. de Castro, C. Juvekar, A. P. Chandrakasan, V. Vaikuntanathan, and A. Joshi, "MAD: Memory-aware design techniques for accelerating fully homomorphic encryption," in *Proc. 56th Annu. IEEE/ACM Int. Symp. Microarchit., MICRO*, Toronto, ON, Canada. New York, NJ, USA: ACM, 2023, pp. 685–697.
- [35] M. A. Will and R. K. L. Ko, "Computing mod without mod," *IACR Cryptol. ePrint Arch.*, 2014, p. 755.
- [36] J. H. Cheon et al., "A full RNS variant of approximate homomorphic encryption," in *Proc. SAC*, 2018, pp. 347–368.
- [37] AMD, *AMD Vitis™ High-Level Synthesis (HLS)*. Accessed: Jan. 8, 2025. [Online]. Available: <https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vitis/vitis-hls.html>
- [38] M. R. Albrecht, "On dual lattice attacks against small-secret LWE and parameter choices in HELib and SEAL," in *Proc. Eurocrypt*, 2017.
- [39] A. Kim, A. Papadimitriou, and Y. Polyakov, "Approximate homomorphic encryption with reduced approximation error," *IACR Cryptol. ePrint Arch.*, 2020, p. 1118.
- [40] A. Brutzkus, O. Elisha, and R. Gilad-Bachrach, "Low latency privacy preserving inference," in *Proc. ICML*, USA, vol. 97, PMLR, 2019, pp. 812–821.



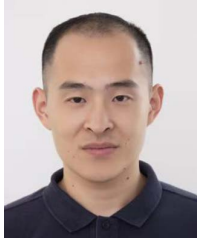
Yilan Zhu received the B.S. degree in 2018 from Shandong University, Jinan, China, where she is currently working toward the Ph.D. degree with the School of Cyber Science and Technology. Her research interests include fully homomorphic encryption and Field Programmable Gate Arrays (FPGAs).



Honghui You received the B.S. degree in 2021 from Shandong University, Jinan, China, where she is currently working toward the Ph.D. degree with the School of Cyber Science and Technology. Her research interests include privacy-preserving machine learning and fully homomorphic encryption.



Qian Lou received the M.S. and Ph.D. degrees from Indiana University Bloomington. Currently, he is an Assistant Professor in computer science, cyber security, and privacy cluster with the University of Central Florida. Prior to joining UCF, he served as a Senior Research Scientist with Samsung Research AI Center. His research interest includes computer vision, natural language processing, and various scientific tasks.



Wei Zhang received the Ph.D. degree from The Hong Kong Polytechnic University, Hong Kong, in 2021. Currently, he is an Associate Professor with the School of Cyber Science and Technology, Shandong University, Qingdao, China. His research interests include design, analysis, and optimization of real time systems and intermittent computing systems.



Shoumeng Yan is the Senior Director of Secure and Trustworthy Computing Business Unit, Ant Group, and the Head of Computing Systems Laboratory of Ant Research. He leads the development of a series of high-profile open-source projects including Occlum TEE OS, HyperEnclave TEE, and Tongsuo crypto library. His teams are also responsible for Ant Linux, NanoVisor Container, and accelerators for privacy preserving computing.



Jiming Xu received the Ph.D. degree from the Memorial University of Newfoundland, Canada. Currently, he is a Senior Engineer majoring in cryptographic algorithms with Ant Group. After his Ph.D., he served as a Joint Postdoctoral Researcher with Peking University Shenzhen Graduate School and the Nations Technologies Inc., Shenzhen. His research interest includes side-channel attacks, cryptographic hardware, post-quantum cryptography, and privacy-preserving computation.



Lei Ju received the Ph.D. degree from the School of Computing, National University of Singapore, Singapore, in 2010. Currently, he is a Professor with the School of Cyber Science and Technology, Shandong University, Jinan, China. His research interests include heterogeneous computing systems and real-time embedded systems.